AVCS Test Outline

AVCS Test Outline

1. Build Integrity Tests

Purpose:
Verify the build is structurally valid and can be meaningfully certified.

Tests:

- artifact location exists
- required files exist
- package/manifest file exists
- environment contract exists
- startup script is declared
- build output is readable
- config references are present
- required entry points exist

Pass condition:
Build is structurally complete and testable.

Fail examples:

- missing `package.json`
- missing env contract
- missing startup declaration
- artifact path invalid

---

2. Functional Correctness Tests

Purpose:
Verify the build actually works across core product behavior.

Tests:

- primary workflow executes successfully
- form submission works
- CRUD actions work where applicable
- API request/response behavior is valid
- expected success state appears
- error state appears correctly
- invalid input is handled correctly
- state transitions complete correctly
- key operator actions complete end-to-end

Pass condition:
Critical workflows and core behaviors execute correctly.

Fail examples:

- submit flow breaks
- data save reports success but does not persist
- endpoint returns invalid payload
- error recovery path traps user

---

3. Security Posture Tests

Purpose:
Verify the build does not expose obvious or material security risk.

Tests:

- auth boundary checks
- permission / role checks
- secret exposure scan
- dependency vulnerability scan
- static security pattern scan
- unsafe config exposure check
- debug/admin surface exposure check
- input handling safety checks
- sensitive logging checks

Pass condition:
No blocker-class security findings and no critical unsafe exposure.

Fail examples:

- auth bypass
- exposed secret
- critical vulnerable dependency
- unsafe debug endpoint exposed

---

4. Performance & Stability Tests

Purpose:
Verify the build performs within acceptable limits and remains stable.

Tests:

- startup/load time test
- route load time test
- repeated action stability test
- resource/budget checks
- latency threshold checks
- performance regression comparison
- light stress / concurrency checks
- long-running repeat-flow stability checks

Pass condition:
Core flows stay within performance thresholds and do not degrade unacceptably.

Fail examples:

- dashboard load too slow
- repeated action causes degradation
- route latency exceeds threshold
- bundle/perf budget exceeded

---

5. Deployment Readiness Tests

Purpose:
Verify the build is actually deployable, not just locally functional.

Tests:

- required env vars documented/present

- startup path works
- deployment config exists
- readiness/health checks exist
- required migration/init step is declared
- deployment assumptions are explicit
- runtime config is valid
- deployment-critical files are present

Pass condition:
The build has enough operational structure to move toward deployment safely.

Fail examples:

- required env var contract missing
- no health/readiness surface
- startup path unclear
- deployment config incomplete

---

6. UI Quality Tests

Purpose:
Verify the interface renders clearly and consistently.

Tests:

- page layout renders correctly
- visual hierarchy is intact
- responsive layout check

- control visibility check
- loading state check
- empty state check
- success state check
- error state visual check
- visual regression comparison

Pass condition:
Important screens render clearly and do not visually break.

Fail examples:

- layout overlap
- hidden CTA
- modal off-screen
- broken responsive state

---

7. UX Workflow Tests

Purpose:
Verify workflows are understandable, completable, and not overly fragile.

Tests:

- primary workflow completion test
- navigation continuity test
- dead-end detection
- recovery-path test

- success-path clarity check
- empty-state actionability check
- error-state recovery usability check
- step-friction review
- workflow efficiency/path sanity check

Pass condition:
Users/operators can complete intended tasks cleanly and coherently.

Fail examples:

- workflow loses context
- user must backtrack through unrelated screens
- no recovery path after failure
- empty state offers no next action

---

8. Accessibility Tests

Purpose:
Verify the build is usable through accessible interaction patterns.

Tests:

- keyboard navigation test
- focus order test
- focus visibility test
- accessible label/role check
- semantic structure check

- form accessibility test
- modal/dialog accessibility test
- contrast/readability check
- primary workflow accessibility test

Pass condition:
Primary workflows are not blocked by accessibility failures.

Fail examples:

- button unreachable by keyboard
- focus trap
- unlabeled form control
- low-contrast critical message

---

9. Enterprise Readiness Tests

Purpose:
Verify the build is supportable, observable, and enterprise-usable beyond just code behavior.

Tests:

- audit trail presence check
- structured logging check
- error observability check
- supportability signal check
- operator clarity check
- role/permission clarity check

- release evidence completeness check
- config explicitness check
- handoff readiness check

Pass condition:
The build is supportable and governable enough for enterprise use.

Fail examples:

- no auditable record for critical action
- error logging too weak
- release evidence incomplete
- role assumptions undocumented

Test Modes Using These Tests

Smoke Run

Uses lightweight checks from:

- Build Integrity
- minimal Functional Correctness
- maybe minimal startup/load check

Focus:
Catch obvious breakage fast.

Functional Run

Uses:

- Functional Correctness tests
- some Build Integrity dependency checks
- workflow completion tests

Focus:
Does the system work?

Security Run

Uses:

- Security Posture tests
- some deployment/config exposure checks

Focus:
Is the build safe enough to release?

Performance Run

Uses:

- Performance & Stability tests
- maybe startup/load checks from Build Integrity

Focus:
Is the build fast and stable enough?

Accessibility Run

Uses:

- Accessibility tests
- some UI/UX surface checks

Focus:
Are primary flows accessible?

Full Certification Run

Uses:

- Build Integrity
- Functional Correctness
- Security Posture
- Performance & Stability
- Deployment Readiness
- later also UI, UX, Accessibility, Enterprise Readiness

Focus:
Full enterprise release judgment.

MVP Test Set

For the version we're building first, the test outline should focus on:

MVP tests

- artifact location exists

- required files exist
- environment contract exists
- startup declaration exists
- primary workflow executes
- key API behavior works
- auth/permission baseline checks
- secret/dependency/config baseline checks
- startup/load timing
- repeated action stability
- deployment config/env readiness checks

That is the core update path you're currently building around.

AVCS Test Catalog (MVP + Core)

AVCS Test Catalog (MVP + Core)

Test ID format

[DOMAIN]-[NUMBER]

Examples:

BI-01
FUNC-04
SEC-02
PERF-03
DEP-01

Domains:

BI Build Integrity
FUNC Functional Correctness
SEC Security Posture
PERF Performance & Stability
DEP Deployment Readiness
UI UI Quality
UX UX Workflow
ACC Accessibility
ENT Enterprise Readiness

---

1. Build Integrity Tests

BI-01 — Artifact Location Exists

Purpose
Ensure the declared build artifact directory exists and is readable.

Fail Conditions

- artifact path invalid
- directory unreadable

Evidence

- filesystem check result

BI-02 — Required Files Present

Purpose
Verify required build files exist.

Examples

- `package.json`
- lock file
- main config

Fail Conditions

- required file missing

BI-03 — Environment Contract Exists

Purpose
Verify env contract surface exists.

Examples

- `.env.example`
- `env.contract.json`
- `.env.schema.json`

Fail Conditions

- environment requirements undocumented

BI-04 — Startup Script Declared

Purpose
Verify the system has a startup path.

Examples

npm start
npm dev
npm build

Fail Conditions

- startup undefined

BI-05 — Build Output Readable

Purpose
Ensure artifact output is accessible.

Fail Conditions

- permission issues
- corrupted build output

---

2. Functional Correctness Tests

FUNC-01 — Primary Workflow Executes

Purpose
Verify the primary user workflow completes.

Example

create project → save → view

Fail Conditions

- workflow breaks
- missing state transition

FUNC-02 — API Contract Valid

Purpose
Ensure API responses match expected shape.

Fail Conditions

- invalid response schema
- missing required fields

FUNC-03 — CRUD Operations Work

Purpose
Verify create, read, update, delete behavior.

Fail Conditions

- operations silently fail
- data persistence failure

FUNC-04 — Error Handling Path Works

Purpose
Ensure the system handles invalid input properly.

Fail Conditions

- silent errors
- unhandled exception

FUNC-05 — State Transition Integrity

Purpose
Ensure system state flows correctly.

Examples

draft → active
pending → approved

Fail Conditions

- illegal transitions
- stuck states

---

3. Security Posture Tests

SEC-01 — Dependency Vulnerability Scan

Purpose
Detect known vulnerable dependencies.

Fail Conditions

- critical CVE present

SEC-02 — Secrets Exposure Scan

Purpose
Detect exposed secrets or credentials.

Fail Conditions

- tokens in code
- credentials in config

SEC-03 — Authentication Boundary Check

Purpose
Ensure authentication is required where expected.

Fail Conditions

- protected route accessible

SEC-04 — Permission Enforcement

Purpose
Ensure roles cannot access unauthorized resources.

Fail Conditions

- role escalation
- bypass

SEC-05 — Unsafe Config Exposure

Purpose
Detect debug/admin exposure.

Fail Conditions

- debug endpoints exposed
- dev configs in production

---

4. Performance & Stability Tests

PERF-01 — Startup Time

Purpose
Measure system startup latency.

Fail Conditions

- exceeds threshold

PERF-02 — Route Load Time

Purpose
Measure page/API load latency.

Fail Conditions

- latency threshold exceeded

PERF-03 — Repeated Action Stability

Purpose
Ensure repeated actions do not degrade performance.

Example

repeat action 100x

Fail Conditions

- memory leak
- latency drift

PERF-04 — Resource Budget Check

Purpose
Verify bundle/resource budgets.

Fail Conditions

- bundle size exceeded
- memory overuse

PERF-05 — Concurrency Stability

Purpose
Verify system stability under concurrent access.

Fail Conditions

- race condition
- crash under load

---

5. Deployment Readiness Tests

DEP-01 — Environment Variables Declared

Purpose
Ensure required env vars are documented.

Fail Conditions

- missing required variable

DEP-02 — Startup Path Works

Purpose
Ensure system can start successfully.

Fail Conditions

- runtime startup failure

DEP-03 — Health Endpoint Exists

Purpose
Ensure health/readiness probe exists.

Fail Conditions

- missing health route

DEP-04 — Deployment Config Present

Purpose
Ensure deployment configuration exists.

Examples

Dockerfile
deployment config
runtime config

Fail Conditions

- deployment undefined

---

6. UI Quality Tests

UI-01 — Layout Render Integrity

Purpose
Ensure UI renders without structural break.

Fail Conditions

- overlapping layout
- broken structure

UI-02 — Responsive Layout Check

Purpose
Ensure responsive behavior works.

Fail Conditions

- mobile layout breaks

UI-03 — Visual Regression Detection

Purpose
Detect unintended visual changes.

Fail Conditions

- screenshot mismatch

---

7. UX Workflow Tests

UX-01 — Workflow Completion Path

Purpose
Verify full workflow path is completable.

Fail Conditions

- dead-end navigation

UX-02 — Error Recovery Path

Purpose
Ensure users can recover from errors.

Fail Conditions

- no recovery path

UX-03 — Empty State Actionability

Purpose
Verify empty states guide user action.

Fail Conditions

- empty state without next step

---

8. Accessibility Tests

ACC-01 — Keyboard Navigation

Purpose
Ensure UI usable with keyboard.

Fail Conditions

- unreachable controls

ACC-02 — Focus Visibility

Purpose
Ensure focus is visible.

Fail Conditions

- hidden focus state

ACC-03 — Accessible Labels

Purpose
Ensure controls have labels.

Fail Conditions

- unlabeled inputs

ACC-04 — Color Contrast

Purpose
Ensure readable contrast.

Fail Conditions

- low contrast critical UI

---

9. Enterprise Readiness Tests

ENT-01 — Audit Trail Presence

Purpose
Ensure critical actions produce audit records.

Fail Conditions

- action not recorded

ENT-02 — Structured Logging

Purpose
Ensure logs are structured and useful.

Fail Conditions

- unstructured logs
- missing context

ENT-03 — Observability Signals

Purpose
Ensure system exposes health/metrics.

Fail Conditions

- no observability signals

MVP Test Set

For the version you are building **right now**, AVCS should start with:

Build Integrity

BI-01
BI-02
BI-03

BI-04

Functional

FUNC-01
FUNC-02
FUNC-03
FUNC-04

Security

SEC-01
SEC-02
SEC-05

Performance

PERF-01
PERF-02
PERF-03

Deployment

DEP-01
DEP-02
DEP-03

Total initial catalog:
~18 core tests

What this enables

Once tests have IDs like this, AVCS can easily:

- attach **findings**
- attach **evidence**
- track **coverage**
- compare **run history**

- map **tools to tests**

Example mapping later:

Lighthouse → PERF-01, PERF-02, ACC-04
k6 → PERF-03, PERF-05
ZAP → SEC-03, SEC-04
Trivy → SEC-01
Playwright → FUNC-01, FUNC-03
BackstopJS → UI-03
axe-core → ACC-01, ACC-02, ACC-03

AVCS Test Execution Engine

AVCS Test Execution Engine

Recommended location:

server/avcs/execution/

Core files:

test-catalog.ts
test-plan-builder.ts
test-runner.ts
tool-adapters/
result-normalizer.ts

---

1. Test Catalog

File:

server/avcs/execution/test-catalog.ts

Purpose
 Single registry of all AVCS tests.

Structure example:

```
export interface AVCSTestDefinition {
 id: string
 domain: string
 name: string
 description: string
 tools: string[]
 runTypes: string[]
 severity: "LOW" | "MEDIUM" | "HIGH" | "CRITICAL"
}
```

Example entries:

```
export const TEST_CATALOG: AVCSTestDefinition[] = [

{
id: "BI-01",
```

domain: "BUILD_INTEGRITY",
name: "Artifact Location Exists",
description: "Verify artifact directory exists and is readable",
tools: ["filesystem"],
runTypes: ["SMOKE","FULL_CERTIFICATION"],
severity: "CRITICAL"
},

{
id: "SEC-01",
domain: "SECURITY",
name: "Dependency Vulnerability Scan",
description: "Detect vulnerable dependencies",
tools: ["trivy","dependency-check"],
runTypes: ["SECURITY","FULL_CERTIFICATION"],
severity: "CRITICAL"
},

{
id: "PERF-02",
domain: "PERFORMANCE",
name: "Route Load Time",
description: "Measure route latency",
tools: ["lighthouse"],
runTypes: ["PERFORMANCE","FULL_CERTIFICATION"],
severity: "HIGH"
}

]

This catalog becomes the **source of truth for AVCS tests**.

---

2. Test Plan Builder

File:

server/avcs/execution/test-plan-builder.ts

Purpose
Convert a **run type → list of tests to execute**.

Example:

```
export function buildTestPlan(runType: string) {

return TEST_CATALOG.filter(test =>
test.runTypes.includes(runType)
)

}
```

Example output for a **FULL_CERTIFICATION run**:

```
[
BI-01
BI-02
BI-03
BI-04

FUNC-01
FUNC-02
FUNC-03

SEC-01
SEC-02
SEC-05

PERF-01
PERF-02
PERF-03

DEP-01
DEP-02
DEP-03
]
```

This becomes the **execution plan**.

---

3. Test Runner

File:

server/avcs/execution/test-runner.ts

Purpose
 Execute tests in order.

Pseudo flow:

```
for (const test of testPlan) {

const adapter = resolveToolAdapter(test.tools)

const result = await adapter.execute(test)

normalizedResults.push(
normalizeResult(test, result)
)

}
```

Output:

AVCSTestResult[]

---

4. Tool Adapter Layer

Folder:

server/avcs/execution/tool-adapters/

Each tool has a wrapper.

Example:

lighthouse-adapter.ts
k6-adapter.ts
zap-adapter.ts
trivy-adapter.ts
playwright-adapter.ts
backstop-adapter.ts
axe-adapter.ts

Adapter interface:

```
export interface AVCSAdapter {
```

```
name: string

execute(test: AVCSTestDefinition): Promise<unknown>

}
```

Example adapter:

```
export class TrivyAdapter implements AVCSAdapter {

name = "trivy"

async execute(test) {

const result = await runTrivyScan()

return result

}

}
```

Adapters isolate **external tool logic**.

---

5. Result Normalizer

File:

server/avcs/execution/result-normalizer.ts

Purpose
Convert tool outputs into AVCS structures.

Input example:

Trivy JSON
Lighthouse JSON
Playwright results

Output example:

AVCSTestResult

Structure:

```
export interface AVCSTestResult {

testId: string

status: "PASS" | "FAIL" | "WARNING"

findings: AVCSFinding[]

evidence: AVCSEvidenceItem[]

}
```

This feeds directly into:

Findings Service
Evidence Service
Coverage Service
Verdict Engine

Execution Flow

Full AVCS execution flow becomes:

```
Build Kit
   |
   ▼
AVCS Run Created
   |
   ▼
Test Plan Builder
   |
   ▼
Test Runner
   |
   ▼
Tool Adapters
   |
   ▼
Result Normalizer
   |
```

Example Run

Example **FULL_CERTIFICATION run**

Plan:

18 tests

Execution:

BI-01 → PASS
BI-02 → PASS
BI-03 → PASS
BI-04 → PASS

FUNC-01 → PASS
FUNC-02 → PASS

SEC-01 → FAIL (dependency CVE)

PERF-02 → WARNING

DEP-01 → PASS
DEP-02 → PASS

Generated artifacts:

Findings: 1 critical
Evidence: 12 items
Coverage: full
Verdict: FAIL

Why this architecture is powerful

This design gives AVCS:

Extensibility

Add a new test:

add test to catalog
add adapter if needed
done

Tool independence

AVCS logic never changes when tools change.

Enterprise traceability

Every test has:

testId
domain
tool
finding
evidence
coverage

Run comparison

Future runs can compare:

PERF-02
before vs after

What you now have

Your AVCS architecture now includes:

Test Catalog
Test Plan Builder
Test Runner
Tool Adapter Layer
Result Normalizer
Findings Service
Evidence Service
Coverage Service
Verdict Engine
History Layer

This is already **a legitimate certification platform architecture**.

AVCS Tool Registry

AVCS Tool Registry

Recommended location:

server/avcs/tools/

Files:

tool-registry.ts
tool-policy.ts
tool-resolver.ts

---

1. Tool Registry (Allowlist)

File:

server/avcs/tools/tool-registry.ts

Purpose
Defines the **official tools AVCS is allowed to run**.

Tool Definition

```
export interface AVCSToolDefinition {
 id: string
 name: string

 category:
   | "SECURITY"
   | "PERFORMANCE"
   | "ACCESSIBILITY"
   | "UI"
   | "FUNCTIONAL"

 officialSource: string

 installMethod:
   | "npm"
   | "docker"
   | "binary"
```

```
 allowedRunTypes: string[]

 domains: string[]

 defaultTimeoutSeconds: number
}
```

Approved Tool List

```
export const TOOL_REGISTRY: AVCSToolDefinition[] = [

{
id: "lighthouse",
name: "Google Lighthouse",
category: "PERFORMANCE",
officialSource: "https://github.com/GoogleChrome/lighthouse",
installMethod: "npm",
allowedRunTypes: ["PERFORMANCE","FULL_CERTIFICATION"],
domains: ["PERFORMANCE","ACCESSIBILITY","UI"],
defaultTimeoutSeconds: 60
},

{
id: "k6",
name: "Grafana k6",
category: "PERFORMANCE",
officialSource: "https://github.com/grafana/k6",
installMethod: "binary",
allowedRunTypes: ["PERFORMANCE","FULL_CERTIFICATION"],
domains: ["PERFORMANCE"],
defaultTimeoutSeconds: 300
},

{
id: "trivy",
name: "Trivy Vulnerability Scanner",
category: "SECURITY",
officialSource: "https://github.com/aquasecurity/trivy",
installMethod: "binary",
allowedRunTypes: ["SECURITY","FULL_CERTIFICATION"],
domains: ["SECURITY","DEPLOYMENT"],
defaultTimeoutSeconds: 120
},
```

{
id: "zap",
name: "OWASP ZAP",
category: "SECURITY",
officialSource: "https://github.com/zaproxy/zaproxy",
installMethod: "docker",
allowedRunTypes: ["SECURITY","FULL_CERTIFICATION"],
domains: ["SECURITY"],
defaultTimeoutSeconds: 300
},

{
id: "semgrep",
name: "Semgrep Community Edition",
category: "SECURITY",
officialSource: "https://github.com/semgrep/semgrep",
installMethod: "binary",
allowedRunTypes: ["SECURITY","FULL_CERTIFICATION"],
domains: ["SECURITY"],
defaultTimeoutSeconds: 120
},

{
id: "playwright",
name: "Playwright",
category: "FUNCTIONAL",
officialSource: "https://github.com/microsoft/playwright",
installMethod: "npm",
allowedRunTypes: ["FUNCTIONAL","FULL_CERTIFICATION"],
domains: ["FUNCTIONAL","UX","UI"],
defaultTimeoutSeconds: 120
},

{
id: "backstop",
name: "BackstopJS",
category: "UI",
officialSource: "https://github.com/garris/BackstopJS",
installMethod: "npm",
allowedRunTypes: ["FULL_CERTIFICATION"],
domains: ["UI"],
defaultTimeoutSeconds: 120
},

```
{
id: "axe",
name: "axe-core",
category: "ACCESSIBILITY",
officialSource: "https://github.com/dequelabs/axe-core",
installMethod: "npm",
allowedRunTypes: ["ACCESSIBILITY","FULL_CERTIFICATION"],
domains: ["ACCESSIBILITY"],
defaultTimeoutSeconds: 60
}

]
```

---

2. Tool Resolver

File:

server/avcs/tools/tool-resolver.ts

Purpose
Ensure only **approved tools** are used.

```
import { TOOL_REGISTRY } from "./tool-registry"

export function resolveTool(toolId: string) {

const tool = TOOL_REGISTRY.find(t => t.id === toolId)

if (!tool) {
throw new Error(`AVCS tool not approved: ${toolId}`)
}

return tool
}
```

Example usage:

```
const tool = resolveTool("trivy")
```

If someone tries:

```
resolveTool("random-scanner")
```

AVCS throws an error.

---

3. Tool Policy

File:

server/avcs/tools/tool-policy.ts

Purpose
 Ensure tools are only run in allowed contexts.

```
export function validateToolForRunType(tool, runType) {

if (!tool.allowedRunTypes.includes(runType)) {

throw new Error(
`Tool ${tool.id} not allowed for run type ${runType}`
)

}

}
```

Example:

```
validateToolForRunType(tool, runType)
```

Integration with Test Runner

Test runner flow becomes:

```
Test
│
▼
Tool Resolver
│
▼
Policy Validation
│
```

▼
Tool Adapter
|
▼
Tool Execution

Example code:

```
const tool = resolveTool(toolId)

validateToolForRunType(tool, runType)

const adapter = adapterFactory(tool.id)

const result = await adapter.execute(test)
```

Safety Benefits

This registry prevents:

Unknown tools

AVCS will reject them.

Malicious tools

Only allowlisted integrations run.

Supply chain surprises

You control the tool sources.

Random adapter execution

Adapters must match registry tools.

Future Enhancements

The registry can later add:

versionPin: string

dockerImage: string

checksum: string

sandboxRequired: boolean

Example:

```
{
id: "trivy",
versionPin: "0.49.0",
dockerImage: "aquasec/trivy:0.49.0"
}
```

This makes AVCS **fully reproducible**.

Current AVCS Execution Stack

Your architecture now looks like:

```
Test Catalog
    │
    ▼
Test Plan Builder
    │
    ▼
Test Runner
    │
    ▼
Tool Registry
    │
    ▼
Tool Resolver
    │
    ▼
```

Tool Adapters
│
▼
Result Normalizer
│
▼
Findings Service
│
▼
Evidence Service
│
▼
Coverage Service
│
▼
Verdict Engine
│
▼
History Layer

This is **very close to a real enterprise verification platform architecture**.

AVCS Coverage Service

AVCS Test Catalog

Test ID Structure

Each test has a stable ID.

Format:

`DOMAIN-NUMBER`

Examples:

- BI-01
- FUNC-02
- SEC-03
- PERF-01

Domain prefixes:

Prefix	Domain
BI	Build Integrity
FUNC	Functional Correctness
SEC	Security Posture
PERF	Performance & Stability
DEP	Deployment Readiness
UI	UI Quality
UX	UX Workflow
ACC	Accessibility
ENT	Enterprise Readiness

---

1. Build Integrity Tests

BI-01 — Artifact Location Exists

Verify the build artifact directory exists and is readable.

Fail examples:

- artifact path invalid
- directory unreadable

BI-02 — Required Files Present

Verify critical build files exist.

Examples:

- package.json
- lock files
- configuration files

Fail examples:

- missing required file
- corrupted file

BI-03 — Environment Contract Exists

Ensure environment variable requirements are documented.

Examples:

- `.env.example`

- `env.contract.json`
- `.env.schema.json`

Fail examples:

- env contract missing
- env variables undocumented

BI-04 — Startup Script Declared

Verify the system has a startup path.

Examples:

- npm start
- npm run dev
- npm run build

Fail examples:

- startup script missing
- startup path invalid

BI-05 — Build Output Readable

Ensure the artifact output can be accessed.

Fail examples:

- permissions block access

- corrupted build output

---

2. Functional Correctness Tests

FUNC-01 — Primary Workflow Executes

Verify the main user workflow completes successfully.

Example:
Create → Save → View

Fail examples:

- workflow crashes
- state transition fails

FUNC-02 — API Contract Valid

Ensure API responses match expected schema.

Fail examples:

- missing required response fields
- invalid response format

FUNC-03 — CRUD Operations Work

Verify create, read, update, delete operations.

Fail examples:

- data not persisted

- operation silently fails

FUNC-04 — Error Handling Path Works

Verify the system handles invalid input properly.

Fail examples:

- unhandled exception
- silent error

FUNC-05 — State Transition Integrity

Verify system states change correctly.

Examples:

- draft → active
- pending → approved

Fail examples:

- illegal transition
- stuck state

---

3. Security Posture Tests

SEC-01 — Dependency Vulnerability Scan

Detect known vulnerable dependencies.

Fail examples:

- critical CVE detected

SEC-02 — Secrets Exposure Scan

Detect exposed secrets.

Fail examples:

- API keys in code
- credentials in config

SEC-03 — Authentication Boundary Check

Ensure protected routes require authentication.

Fail examples:

- unauthorized access possible

SEC-04 — Permission Enforcement

Ensure roles cannot access unauthorized resources.

Fail examples:

- role escalation
- permission bypass

SEC-05 — Unsafe Config Exposure

Detect exposed debug or admin surfaces.

Fail examples:

- debug endpoints exposed
- dev configs in production

---

4. Performance & Stability Tests

PERF-01 — Startup Time

Measure application startup latency.

Fail examples:

- startup exceeds threshold

PERF-02 — Route Load Time

Measure page or API load latency.

Fail examples:

- route latency exceeds threshold

PERF-03 — Repeated Action Stability

Ensure repeated operations remain stable.

Example:
Repeat action 100 times.

Fail examples:

- performance degradation
- memory leak

PERF-04 — Resource Budget Check

Verify resource usage stays within limits.

Fail examples:

- bundle size too large
- excessive memory usage

PERF-05 — Concurrency Stability

Verify system stability under concurrent users.

Fail examples:

- race conditions
- crashes under load

---

5. Deployment Readiness Tests

DEP-01 — Environment Variables Declared

Verify required environment variables are defined.

Fail examples:

- missing required variable

DEP-02 — Startup Path Works

Verify the application starts successfully.

Fail examples:

- runtime startup failure

DEP-03 — Health Endpoint Exists

Ensure readiness or health endpoint exists.

Fail examples:

- missing health check

DEP-04 — Deployment Config Present

Verify deployment configuration exists.

Examples:

- Dockerfile
- deployment configuration

Fail examples:

- deployment undefined

---

6. UI Quality Tests

UI-01 — Layout Render Integrity

Verify UI layout renders correctly.

Fail examples:

- overlapping UI
- broken layout

UI-02 — Responsive Layout Check

Verify responsive behavior works.

Fail examples:

- mobile layout broken

UI-03 — Visual Regression Detection

Detect unintended visual changes.

Fail examples:

- screenshot mismatch

---

7. UX Workflow Tests

UX-01 — Workflow Completion Path

Verify workflows can be completed.

Fail examples:

- dead-end navigation

UX-02 — Error Recovery Path

Ensure users can recover from errors.

Fail examples:

- no recovery option

UX-03 — Empty State Actionability

Ensure empty states guide the user.

Fail examples:

- empty state with no action

---

8. Accessibility Tests

ACC-01 — Keyboard Navigation

Ensure UI can be navigated via keyboard.

Fail examples:

- unreachable controls

ACC-02 — Focus Visibility

Ensure focus indicators are visible.

Fail examples:

- hidden focus state

ACC-03 — Accessible Labels

Ensure form controls have labels.

Fail examples:

- unlabeled inputs

ACC-04 — Color Contrast

Ensure readable UI contrast.

Fail examples:

- low contrast on critical text

---

9. Enterprise Readiness Tests

ENT-01 — Audit Trail Presence

Verify critical actions are recorded.

Fail examples:

- action not logged

ENT-02 — Structured Logging

Ensure logs are structured and usable.

Fail examples:

- missing context
- unstructured logs

ENT-03 — Observability Signals

Ensure system exposes monitoring signals.

Fail examples:

- no health metrics
- no telemetry

MVP Test Set (Initial AVCS Implementation)

The first version of AVCS should focus on these tests:

Build Integrity

- BI-01
- BI-02
- BI-03
- BI-04

Functional

- FUNC-01
- FUNC-02
- FUNC-03
- FUNC-04

Security

- SEC-01
- SEC-02
- SEC-05

Performance

- PERF-01
- PERF-02
- PERF-03

Deployment

- DEP-01
- DEP-02
- DEP-03

Total initial tests: **18**

AVCS Test → Tool Mapping

AVCS Test → Tool Mapping

Build Integrity

These are mostly **internal AVCS checks**, not external tools.

- **BI-01 Artifact Location Exists** → internal filesystem check
- **BI-02 Required Files Present** → internal filesystem check
- **BI-03 Environment Contract Exists** → internal config/file check
- **BI-04 Startup Script Declared** → internal package/config parser
- **BI-05 Build Output Readable** → internal filesystem/permission check

Why:
These are structural checks on the build artifact itself. External audit tools are not the best fit here.

Functional Correctness

Use **Playwright** as the main tool. It is built for end-to-end browser automation and fits user-flow validation well.

- **FUNC-01 Primary Workflow Executes** → Playwright
- **FUNC-02 API Contract Valid** → Playwright + internal response/schema assertions
- **FUNC-03 CRUD Operations Work** → Playwright
- **FUNC-04 Error Handling Path Works** → Playwright
- **FUNC-05 State Transition Integrity** → Playwright + internal state assertions

Best role in AVCS:
Functional Run, Full Certification Run

Security Posture

Use **Trivy, ZAP, Semgrep, and OWASP Dependency-Check** together, because they cover different security surfaces: Trivy for vulnerabilities/misconfigurations, ZAP for web app scanning, Semgrep for static code patterns, and Dependency-Check for known vulnerable dependencies.

- **SEC-01 Dependency Vulnerability Scan** → Trivy, OWASP Dependency-Check
- **SEC-02 Secrets Exposure Scan** → Trivy
- **SEC-03 Authentication Boundary Check** → ZAP, Playwright
- **SEC-04 Permission Enforcement** → ZAP, Playwright
- **SEC-05 Unsafe Config Exposure** → Trivy, ZAP, Semgrep

Best role in AVCS:
Security Run, Full Certification Run, Pre-Deployment Gate Run

---

Performance & Stability

Use **Lighthouse** for web performance audits and **k6** for load/stability. Locust is also valid if you want Python-scripted traffic, but k6 is the cleaner first choice. Lighthouse supports CLI/Node/CI workflows, and k6 is purpose-built for performance/load testing.

- **PERF-01 Startup Time** → Lighthouse
- **PERF-02 Route Load Time** → Lighthouse
- **PERF-03 Repeated Action Stability** → k6, Playwright
- **PERF-04 Resource Budget Check** → Lighthouse
- **PERF-05 Concurrency Stability** → k6

Best role in AVCS:
Performance Run, Full Certification Run

Deployment Readiness

These are mostly **internal AVCS checks**, with some security-tool support.

- **DEP-01 Environment Variables Declared** → internal config/env contract check
- **DEP-02 Startup Path Works** → internal startup execution check
- **DEP-03 Health Endpoint Exists** → internal endpoint check, optionally Playwright/API probe
- **DEP-04 Deployment Config Present** → internal config/file check
- **DEP support checks for misconfig/vulns** → Trivy

Best role in AVCS:
Full Certification Run, Pre-Deployment Gate Run

UI Quality

Use **BackstopJS** for visual regression and **Playwright** for UI flow rendering checks. BackstopJS is specifically for catching visual changes, while Playwright is better for behavior and screen-state automation. Lighthouse can contribute some UI-related quality signals too.

- **UI-01 Layout Render Integrity** → Playwright, BackstopJS
- **UI-02 Responsive Layout Check** → Playwright, BackstopJS
- **UI-03 Visual Regression Detection** → BackstopJS

Best role in AVCS:
UI/UX Run, Full Certification Run

UX Workflow

Use **Playwright** as the main tool.

- **UX-01 Workflow Completion Path** → Playwright
- **UX-02 Error Recovery Path** → Playwright
- **UX-03 Empty State Actionability** → Playwright

Best role in AVCS:
UI/UX Run, Full Certification Run

Important note:
UX is only partly automatable. Playwright can prove flow completion and recovery behavior, but human review or AVCS rubric scoring may still be needed for deeper UX judgment.

Accessibility

Use **axe-core** and **Pa11y**. Lighthouse also contributes accessibility audits, but axe-core should be the primary engine because it is specifically an automated accessibility engine, and Pa11y is purpose-built for accessibility testing in CLI/CI workflows.

- **ACC-01 Keyboard Navigation** → axe-core, Playwright
- **ACC-02 Focus Visibility** → axe-core, Playwright
- **ACC-03 Accessible Labels** → axe-core, Pa11y
- **ACC-04 Color Contrast** → axe-core, Pa11y, Lighthouse

Best role in AVCS:
Accessibility Run, Full Certification Run

Enterprise Readiness

Mostly **internal AVCS checks**, plus a few supporting tools.

- **ENT-01 Audit Trail Presence** → internal AVCS audit/log validation

- **ENT-02 Structured Logging** → internal log-shape validation
- **ENT-03 Observability Signals** → internal health/metrics validation
- **ENT support from security/config scanning** → Trivy, Semgrep

Best role in AVCS:
Full Certification Run, Pre-Deployment Gate Run

Recommended MVP Mapping

If you want the smallest strong first integration set, use this:

- **Playwright** → FUNC-01, FUNC-03, FUNC-04, UX-01, UX-02
- **Trivy** → SEC-01, SEC-02, SEC-05, DEP support
- **OWASP Dependency-Check** → SEC-01
- **Semgrep** → SEC-05
- **ZAP** → SEC-03, SEC-04
- **Lighthouse** → PERF-01, PERF-02, PERF-04, ACC-04
- **k6** → PERF-03, PERF-05
- **BackstopJS** → UI-03
- **axe-core** → ACC-01, ACC-02, ACC-03, ACC-04
- **Pa11y** → ACC-03, ACC-04

That gives AVCS strong coverage without overloading the first implementation. The safer installation discipline is to use official sources only; Trivy's docs explicitly distinguish official and community installation methods, which is a good model for your tool policy.

Best way to implement this in AVCS

Use a structure like:

- **internal checks only** for BI, most DEP, most ENT
- **Playwright spine** for FUNC, UX, some UI, some ACC
- **security lane** with Trivy + Dependency-Check + Semgrep + ZAP
- **performance lane** with Lighthouse + k6
- **visual lane** with BackstopJS
- **accessibility lane** with axe-core + Pa11y + Lighthouse

That keeps AVCS clean:

- internal checks for artifact truth
- external tools for specialized audits
- AVCS adapters to normalize everything into findings, evidence, coverage, and verdicts

AVCS integration matrix

Here's the **AVCS integration matrix by test ID**.

Tool roles are based on the official project scope for each tool: Playwright for browser automation, Lighthouse for web quality/performance/accessibility audits, k6 for load testing, ZAP for web security testing, Trivy for vulnerability/misconfiguration scanning, Dependency-Check for vulnerable dependencies, Semgrep for static analysis, BackstopJS for visual regression, and axe-core / Pa11y for accessibility.

AVCS Integration Matrix

Test ID	Test Name	Primary Tool(s)	Run Type(s)	Evidence Output	Phase
BI-01	Artifact Location Exists	Internal AVCS check	Smoke, Full Certification	filesystem result, summary	MVP
BI-02	Required Files Present	Internal AVCS check	Smoke, Full Certification	file check result, summary	MVP
BI-03	Environment Contract Exists	Internal AVCS check	Smoke, Full Certification	config/file check result	MVP
BI-04	Startup Script Declared	Internal AVCS check	Smoke, Full Certification	parsed package/config result	MVP
BI-05	Build Output Readable	Internal AVCS check	Smoke, Full Certification	filesystem/permission result	Later
FUNC-01	Primary Workflow Executes	Playwright	Functional, Full Certification	run log, screenshots, flow result	MVP
FUNC-02	API Contract Valid	Playwright + internal assertions	Functional, Full Certification	request/response capture, assertion result	MVP
FUNC-03	CRUD Operations Work	Playwright	Functional, Full Certification	flow log, screenshots, state result	MVP
FUNC-04	Error Handling Path Works	Playwright	Functional, Full Certification	error-state screenshots, logs	MVP

FUNC-05	State Transition Integrity	Playwright + internal assertions	Functional, Full Certification	state transition records	Later
SEC-01	Dependency Vulnerability Scan	Trivy, Dependency-Check	Security, Full Certification	scan reports, CVE results	MVP
SEC-02	Secrets Exposure Scan	Trivy	Security, Full Certification	secret scan report	MVP
SEC-03	Authentication Boundary Check	ZAP, Playwright	Security, Full Certification	auth test result, findings, traces	Later
SEC-04	Permission Enforcement	ZAP, Playwright	Security, Full Certification	permission test result, findings	Later
SEC-05	Unsafe Config Exposure	Trivy, Semgrep, ZAP	Security, Full Certification	scan report, config findings	MVP
PERF-01	Startup Time	Lighthouse	Performance, Full Certification	audit JSON, score, timings	MVP
PERF-02	Route Load Time	Lighthouse	Performance, Full Certification	audit JSON, route timings	MVP
PERF-03	Repeated Action Stability	k6, Playwright	Performance, Full Certification	load metrics, stability logs	MVP
PERF-04	Resource Budget Check	Lighthouse	Performance, Full Certification	bundle/resource audit	Later
PERF-05	Concurrency Stability	k6	Performance, Full Certification	load report, latency/error metrics	Later
DEP-01	Environment Variables Declared	Internal AVCS check	Full Certification, Pre-Deployment Gate	env contract check result	MVP

DEP-02	Startup Path Works	Internal AVCS check	Full Certification, Pre-Deployment Gate	startup execution log	MVP
DEP-03	Health Endpoint Exists	Internal AVCS check, optional Playwright/API probe	Full Certification, Pre-Deployment Gate	endpoint probe result	MVP
DEP-04	Deployment Config Present	Internal AVCS check, Trivy support	Full Certification, Pre-Deployment Gate	config check, scan result	Later
UI-01	Layout Render Integrity	Playwright, BackstopJS	UI/UX, Full Certification	screenshots, visual diff	Later
UI-02	Responsive Layout Check	Playwright, BackstopJS	UI/UX, Full Certification	viewport screenshots, diffs	Later
UI-03	Visual Regression Detection	BackstopJS	UI/UX, Full Certification	baseline diff report	Later
UX-01	Workflow Completion Path	Playwright	UI/UX, Full Certification	flow result, screenshots	Later
UX-02	Error Recovery Path	Playwright	UI/UX, Full Certification	recovery flow log, screenshots	Later
UX-03	Empty State Actionability	Playwright	UI/UX, Full Certification	state screenshot, flow result	Later
ACC-01	Keyboard Navigation	axe-core, Playwright	Accessibility, Full Certification	accessibility report, flow result	Later
ACC-02	Focus Visibility	axe-core, Playwright	Accessibility, Full Certification	accessibility report, screenshots	Later

ACC-03	Accessible Labels	axe-core, Pa11y	Accessibility, Full Certification	accessibility report	Later
ACC-04	Color Contrast	axe-core, Pa11y, Lighthouse	Accessibility, Full Certification	accessibility report, audit result	Later
ENT-01	Audit Trail Presence	Internal AVCS check	Full Certification, Pre-Deployment Gate	audit/log validation result	Later
ENT-02	Structured Logging	Internal AVCS check	Full Certification, Pre-Deployment Gate	log-shape validation result	Later
ENT-03	Observability Signals	Internal AVCS check	Full Certification, Pre-Deployment Gate	health/metrics validation result	Later

Best MVP stack

For the first strong version, use:

- **Internal AVCS checks** for BI and DEP
- **Playwright** for FUNC
- **Trivy + Dependency-Check + Semgrep** for core SEC
- **Lighthouse + k6** for PERF

That gives you a practical MVP without overloading the first release. The tool fit is strong for those categories based on the official docs.

Clean next step

The next thing to make is the **tool adapter plan**, meaning one adapter outline for:

- Playwright
- Lighthouse
- Trivy
- Dependency-Check
- Semgrep
- k6